Farmy: A Personalized Farm-Memory Architecture for Smallholder Agricultural Support

Doan Trong Luc
Faculty of Software Engineering
FPT University
Da Nang City, Vietnam
lucdtse173180@fpt.edu.vn

Vo Ha Dong
Faculty of Software Engineering
FPT University
Da Nang City, Vietnam
dongvhse@fpt.edu.vn

Nguyen Vu Hoang
Faculty of Software Engineering
FPT University
Da Nang City, Vietnam
hoangnvse@fpt.edu.vn

Le Bao Nhi
Faculty of Software Engineering
FPT University
Da Nang City, Vietnam
nhibtse@fpt.edu.vn

***Abstract*—Smallholder-oriented digital agriculture tools often separate record keeping, disease recognition, and advisory support into independent functions. As a result, recommendations are rarely grounded in the history of a particular farm. This paper presents the design of Farmy, a Progressive Web Application that treats farmers' diary records as reusable operational memory. The proposed architecture combines two retrieval sources—curated agricultural documents and user-authorized diary entries—to construct context for a conversational assistant. It further integrates image-based plant-disease support, asynchronous embedding, rule-based reminders, and an opt-in mechanism for contributing de-identified field images. MongoDB is used as the authoritative application datastore, while pgvector is restricted to a rebuildable similarity-search index. The paper contributes a tenant-isolated dual-corpus retrieval design, a versioned Prompt-as-Code component, and an evaluation protocol that separates retrieval quality, answer groundedness, diagnostic usefulness, safety, and usability. Because the implementation is at prototype stage, the paper reports architectural rationale and testable hypotheses rather than claiming clinical, agronomic, or production effectiveness. The design is intended to support subsequent empirical evaluation with Vietnamese smallholder users and qualified agricultural reviewers.**

***Index Terms*—Agricultural AI, Retrieval-Augmented Generation, Crop Disease Detection, Personalized Recommendation, Progressive Web Application, Prompt Engineering, Gemini Vision, pgvector, Smallholder Farming, Vietnam**

I. Introduction

A. Motivation

Agricultural decisions are cumulative. A recommendation that is reasonable in isolation may be inappropriate when a field was recently treated, when symptoms have repeatedly appeared, or when weather and growth stage differ from the assumed conditions. Yet many smallholder farmers still maintain operational history through memory, paper notes, or fragmented messaging conversations. This makes past actions difficult to search and limits the ability of digital advisory systems to provide farm-specific guidance.

Vietnam provides a relevant setting for this problem. Agriculture remains an important source of employment and rural livelihood, while smartphone-based services create an increasingly practical channel for decision support [1]. However, a useful system must work within constraints that are often secondary in general-purpose applications: intermittent connectivity, low-cost devices, short interaction windows, Vietnamese terminology, and high consequences when advice concerns pesticides or disease management.

B. Research Gap

Existing agricultural applications commonly address one of three tasks: recording farm activities, recognizing visual symptoms, or delivering general recommendations. Treating these tasks independently creates two limitations. First, advisory responses cannot use the farmer's own operational history unless that history is manually repeated in every query. Second, image-based diagnosis is often presented as an isolated result rather than an event that should update the farm record, trigger follow-up tasks, and remain available for later discussion.

This work investigates whether a lightweight farm diary can serve as a personal retrieval corpus alongside a curated agricultural knowledge base. The central premise is that advisory quality depends not only on access to domain documents, but also on controlled access to the specific user's prior actions and observations. The design therefore emphasizes data provenance, tenant isolation, explicit uncertainty, and separation between model-generated text and application-enforced safety controls.

C. Research Questions

The paper is organized around the following research questions:

1) How can a retrieval pipeline combine shared agricultural knowledge with private farm history without exposing one user's records to another?
2) How can prompt construction, citations, and AI outputs be structured so that they are testable and auditable

rather than embedded as informal strings in business logic?
3) How can image-based plant-disease support be integrated with diaries and reminders while communicating uncertainty and avoiding unsupported confidence claims?
4) Which evaluation protocol can distinguish retrieval quality, response groundedness, safety, usability, and system performance in a prototype-stage agricultural assistant?

D. Contributions

The paper makes four design contributions:

- a **tenant-isolated dual-corpus RAG architecture** that retrieves from a curated knowledge collection and the authenticated user's diary memory under explicit source and ownership constraints;
- a **versioned Prompt-as-Code design** that separates instructions, untrusted retrieved content, user input, and application-generated citation metadata;
- an **integrated plant-observation workflow** in which image analysis can be linked to a diary entry, follow-up reminders, and subsequent chat context while retaining uncertainty and provenance; and
- a **staged evaluation framework** that defines measurable outcomes without presenting planned experiments as completed results.

The contribution is architectural rather than algorithmic: Farmy does not introduce a new foundation model or vision classifier. Its novelty lies in coordinating farm memory, retrieval, safety controls, and user workflows into a single testable system for a smallholder context.

II. Related Work

A. Digital Agriculture and Farm Records

Digital agriculture research has shown that data-driven systems can support crop monitoring, decision making, and resource management, but adoption depends on cost, connectivity, usability, and the fit between software workflows and local farming practice [3]. Commercial farm-management platforms are generally optimized for structured operations and larger farms. In contrast, the present work focuses on a diary-first interaction model in which short records, photographs, and completed tasks gradually form a searchable farm history.

B. Retrieval-Augmented Agricultural Assistance

Retrieval-Augmented Generation (RAG) combines parametric language models with external documents, allowing generated answers to be conditioned on retrieved evidence [4]. Subsequent work has explored retrieval timing, document grounding, and active retrieval for knowledge-intensive generation [14]. Agricultural systems have begun applying RAG to domain question answering, including crop-oriented assistants and region-aware advisory pipelines [15], [16]. These studies reinforce the value of grounding advice in verified sources, but most treat retrieval as a shared-domain knowledge problem. Farmy adds a second, private corpus: the authenticated farmer's operational diary. The resulting challenge is not only relevance, but also ownership filtering, provenance, recency, and safe merging of heterogeneous evidence.

C. Plant-Disease Recognition in Field Conditions

PlantVillage enabled large-scale study of image-based disease classification under comparatively controlled image conditions [5], [6]. PlantDoc later highlighted the greater variability of in-the-wild images, including cluttered backgrounds, uneven illumination, and diverse viewpoints [7]. EfficientNet and MobileNetV3 offer useful efficiency-accuracy trade-offs for future task-specific deployment [8], [9]. Farmy does not claim to replace plant pathologists. The prototype instead uses a multimodal model as a structured observation assistant, preceded by file and image-quality checks and followed by application-level warnings and review pathways.

D. Positioning of This Work

The proposed system sits at the intersection of three strands: farm record management, retrieval-grounded conversational assistance, and image-supported plant observation. Its distinguishing feature is the use of personal diary history as controlled retrieval memory and the propagation of a plant observation across multiple workflows. Unlike work centered on a new model or benchmark, this paper concentrates on system boundaries, data ownership, provenance, and an evaluation design suitable for determining whether the integrated approach produces practical value.

III. Proposed Method

This section formalizes the proposed workflow and the design choices that can later be evaluated experimentally. The emphasis is on controlled retrieval, traceable prompt construction, and integration between observations and farm memory. The term "proposed" is used deliberately: performance claims require the evaluation described in Section IX.

A. Problem Formulation

1) Farm Diary Model: Let $\mathcal{U}$ denote the set of registered farmers. Each farmer $u \in \mathcal{U}$ maintains a personal diary $\mathcal{D}_u = \{d_1, d_2, \ldots, d_n\}$ ordered by timestamp, where each entry d_i is a structured tuple:

$$d_i = \langle c_i,\ g_i,\ \tau_i,\ t_i,\ w_i,\ \mathbf{P}_i \rangle \tag{1}$$

in which $c_i \in \mathcal{C}$ is the crop type (e.g., *lúa*, *bắp*), g_i is the growth stage, τ_i is the UTC timestamp, t_i is the free-text note, w_i is the weather context (temperature, humidity, rainfall), and $\mathbf{P}_i$ is the set of associated photographs.

2) Three Advisory Sub-Problems: Prior work [4], [15] treats agricultural advisory as a single-corpus retrieval task over a shared knowledge base $\mathcal{K}$, producing responses that are identical for all farmers querying the same question. Farmy decomposes the problem into three sub-problems that jointly require *both* domain knowledge and personal farm context:

P1 — Personalized Advisory (PA). Given query q from farmer u, generate response r grounded in $\mathcal{K}$ *and* $\mathcal{D}_u$:

$$r = f_{\text{LLM}}\big(q,\ \mathcal{C}_{\text{hybrid}}(q, u)\big) \tag{2}$$

P2 — Visual Disease Detection (DD). Given photograph $p \in \mathbf{P}_i$, predict structured diagnosis:

$$\hat{y} = f_{\text{vision}}(V(p)) \tag{3}$$

where $V(\cdot)$ denotes the upstream validation pipeline (Section III-D).

P3 — Weekly Insight Synthesis (IS). Given the seven-day diary window $\mathcal{D}_u^{(7)} = \{d_i \mid \tau_i \geq \tau_{\text{now}} - 7\text{d}\}$, generate a forward-looking farm report:

$$s_u = f_{\text{LLM}}\big(\mathcal{C}_{\text{hybrid}}(\mathcal{D}_u^{(7)}, u)\big) \tag{4}$$

The key distinction from prior work is the shared context function $\mathcal{C}_{\text{hybrid}}$, which retrieves from *two* corpora simultaneously. This is described in Section III-B.

B. Hybrid RAG Architecture

1) Motivation: A shared agricultural corpus can explain diseases, cultivation practices, and safety principles, but it cannot answer questions that depend on a specific farm's prior actions. Conversely, diary records provide local context but are not authoritative agronomic references. The proposed retrieval function therefore treats these corpora as complementary rather than interchangeable. Knowledge chunks provide externally curated evidence, while diary chunks provide user-owned episodic context. Their provenance remains visible throughout retrieval and response construction.

For example, a system may retrieve a knowledge passage describing conditions associated with brown planthopper outbreaks and, separately, a diary entry recording a recent intervention. The assistant may use both items to frame follow-up questions, but it must not infer that treatment should be repeated solely from temporal proximity. Such decisions require current symptoms, label instructions, and potentially expert review. This distinction prevents a personalized answer from being mistaken for an automatically validated prescription.

2) Retrieval Algorithm: Let $\phi : \Sigma^* \to \mathbb{R}^{768}$ denote the `text-embedding-004` embedding function. The context assembly procedure $\mathcal{C}_{\text{hybrid}}$ proceeds as follows:

1) Compute query embedding: $\mathbf{e}_q = \phi(q) \in \mathbb{R}^{768}$.
2) Retrieve top-k candidates from each corpus via Approximate Nearest Neighbour (ANN) search with cosine similarity $\text{sim}(\cdot,\cdot)$, minimum threshold $\theta = 0.5$:

$$R_{\mathcal{K}} = \text{ANN}(\mathbf{e}_q,\ \mathcal{K},\ k{=}10,\ \theta) \tag{5}$$

$$R_{\mathcal{D}} = \text{ANN}(\mathbf{e}_q,\ \mathcal{D}_u,\ k{=}10,\ \theta) \tag{6}$$

3) Merge and rank by descending similarity score:

$$R = \text{sort}_{\downarrow}\big(R_{\mathcal{K}} \cup R_{\mathcal{D}},\ \text{sim}\big) \tag{7}$$

4) Fetch full document content from MongoDB using $\{\, r.\texttt{source\_id} \mid r \in R \,\}$. *Note: pgvector stores only vector indices; all text content resides in MongoDB.*
5) Assemble context string $\mathcal{C}$ under budget $B = 6{,}000$ characters, ordered by descending score.

ANN search uses a pgvector HNSW index with initial parameters $m{=}16$ and ef_construction$=64$. These values are configuration defaults rather than verified performance guarantees and must be benchmarked on the deployed dataset and infrastructure. Only rows with `is_active = TRUE` are considered. Diary-memory retrieval additionally enforces the authenticated-user invariant `metadata.userId = authenticatedUserId`; this filter is applied by application code and cannot be supplied or overridden by the model.

3) Text Chunking Policy: Raw diary notes are segmented before embedding. Let $\ell(\cdot)$ denote character length after whitespace normalization:

$$\text{chunk}(t) = \begin{cases} \emptyset, & \ell(t) < 20, \\ \{t\}, & 20 \leq \ell(t) < 500, \\ \text{slide}(t), & \ell(t) \geq 500, \end{cases} \tag{8}$$

where the diary configuration is $w = 300$, $s = 100$, and $m = 10$.

where $\text{slide}(t, w, s, m)$ produces up to m overlapping windows of width w and step s. Knowledge documents use $w{=}500$, $s{=}150$, $m{=}50$ to preserve argumentative context across longer passages. The 20-character floor eliminates semantically vacuous entries (e.g., single-word notes) that would add noise to the index.

4) Embedding Lifecycle: Embeddings are written asynchronously via BullMQ to decouple latency-sensitive diary creation from the embedding API call. The **Dual Write** invariant is:

$$\forall\, d_i \in \mathcal{D}_u :\ d_i \in \text{MongoDB} \implies \exists\ \text{BullMQ job for } \phi(d_i) \tag{9}$$

If the embedding job fails permanently after 3 exponential-backoff retries, d_i remains safely in MongoDB; only its search coverage is temporarily reduced. A nightly gap-fill cron requeues any entries without active embeddings, bounding the maximum staleness window to 24 hours.

On diary update, the replacement embeddings are generated first and activated atomically with deactivation of the previous rows where supported by the index store. If generation fails, the previous active vectors remain available. Inactive rows are removed later by a cleanup job. This ordering avoids a temporary loss of retrieval coverage.

C. Prompt-as-Code Engineering

1) Design Rationale: Existing LLM-based agricultural systems embed prompt strings directly in business logic, making them untestable and prone to silent regressions when templates change. Farmy AI introduces **Prompt-as-Code**: prompt construction is isolated in a dedicated `PromptModule` that accepts structured inputs and returns a versioned `BuiltPrompt` record:

$$\texttt{BuiltPrompt} = \langle\, \pi_{\text{sys}},\ \pi_{\text{usr}},\ v,\ \tau_{\text{build}} \,\rangle \tag{10}$$

where π_{sys} is the system prompt, π_{usr} is the user-turn content, v is the semantic version string, and τ_{build} is the build timestamp logged for regression detection.

2) Determinism Guarantee: Given identical inputs $(q, \mathcal{C}, h, m_{\text{pet}})$ — query, context, conversation history, and pet mood — the builder produces byte-identical output. Non-deterministic primitives (`Date.now()`, `Math.random()`) are *prohibited* inside builder functions. This guarantee enables snapshot-based unit testing. The planned suite covers all three builder methods (`buildChatPrompt`, `buildVisionPrompt`, and `buildWeeklyInsightPrompt`); test counts and coverage percentages are reported only after they are produced by the implemented codebase and CI pipeline.

3) Instruction and Data Separation: Retrieved documents and diary text are treated as untrusted data, not as executable instructions. Prompt construction places system rules, retrieved evidence, conversation history, and the current user message in separately labeled slots. A lightweight sanitizer removes malformed control tokens and known prompt-injection markers, but this mechanism is not considered a complete defense because paraphrases and indirect instructions cannot be reliably blocked by pattern matching alone.

Security-sensitive behavior is therefore enforced outside the model. Tenant filters are added by the repository layer, tool permissions are fixed by application code, output must satisfy a typed schema, and source identifiers are attached from retrieval results rather than accepted from generated text. Adversarial tests will include direct instruction override attempts, malicious diary content, and hostile text embedded in knowledge documents.

4) Slot Truncation: Hard character limits prevent context-window overflow while preserving the highest-relevance content:

TABLE I
PROMPT SLOT TRUNCATION LIMITS

Slot	Char Limit
User message	2 000
RAG context	6 000
Conversation history	4 000 (last 6 turns)

D. Visual Disease Detection Pipeline

1) Upstream Validation: To minimize Gemini Vision API calls and enforce minimum image quality, four sequential gates are applied before model invocation (Figure 2). Gate 1 enforces per-user daily quotas via Redis counters (3 scans/day free; 10 scans/day premium). Gate 2 inspects binary magic bytes using the `file-type` library, independently of the client-declared MIME type, to block format spoofing. Gate 3 computes the Laplacian variance σ_L^2 via Sharp.js and rejects images with $\sigma_L^2 < 100$, calibrated empirically on field images rated unusable by agronomist reviewers. Gate 4 checks perceptual hash (pHash) similarity: images within Hamming distance $\delta_H < 10$ of a cached scan within 7 days return the cached result, eliminating duplicate API calls.

2) Structured Output Contract: The vision component returns a typed observation rather than an unconstrained diagnosis:

$$\hat{y} = \langle p,\, C,\, q,\, u,\, S,\, N,\, W \rangle, \tag{11}$$

where p indicates whether plant material is visible; C contains candidate conditions; q and u denote evidence quality and ordinal certainty; and S, N, and W contain symptoms, next steps, and warnings.

evidence_quality summarizes whether the image is sufficiently clear and complete; *certainty* is an ordinal self-assessment (low, medium, or high), not a calibrated probability. Multiple candidate conditions may be returned when symptoms are visually ambiguous. Low-quality or low-certainty outputs require a retake request or referral to a qualified agricultural officer. Chemical names, dosage, and pre-harvest intervals are not accepted as authoritative solely because they appear in model output.

3) Post-Processing Safety Controls: Application code validates the output schema and applies deterministic checks to treatment-related content. Where a maintained regulatory dataset is available, referenced active ingredients can be compared against current Vietnamese plant-protection restrictions. A match triggers a visible warning and prevents the result from being presented as a routine recommendation. This control reduces risk but does not establish agronomic correctness or legal compliance; regulatory data must be versioned, reviewed, and updated by an accountable maintainer. All high-risk outputs direct the user to product labels and qualified local guidance.

E. Cost-Constrained AI Infrastructure

A core design goal of Farmy is to minimize infrastructure expenditure during the MVP phase. The architecture uses free or low-cost service tiers where available, but does not assume that quotas, prices, service-level guarantees, or eligibility remain unchanged. Three mechanisms support this cost-constrained deployment.

1) Gemini Free-Tier Quota Management: LLMModule maintains provider-specific Redis counters using configurable limits loaded from deployment settings. When the configured generation quota is saturated, interactive chat returns a graceful fallback, while background jobs receive a rate-limit exception and retry with bounded exponential backoff. Concrete quota values are verified against the provider documentation at deployment time rather than hard-coded as research claims.

2) pgvector over Atlas Vector Search: pgvector with HNSW was selected as a separately deployable, rebuildable search index because it provides explicit control over indexing and can be operated independently from the MongoDB business datastore. No fixed latency or price advantage is assumed without a deployment-specific benchmark and a current provider-cost review. The separation of vector index (pgvector)

from document store (MongoDB) preserves recoverability: if the pgvector index is lost, it can be fully reconstructed by re-running the embedding pipeline over MongoDB.

3) Delay Spreading for Batch Insight Jobs: The weekly insight generator must process all N users within a Sunday morning window without exceeding the 15 RPM free-tier quota. A **Delay Spreading Algorithm** assigns each user u_i a BullMQ job delay:

$$\Delta_i = \left\lfloor \frac{i}{N} \times T_{\text{win}} \right\rfloor, \quad T_{\text{win}} = 14{,}400\,\text{s} \tag{12}$$

This distributes jobs uniformly over four hours. It reduces burstiness, but does not by itself guarantee quota compliance for arbitrary N; worker concurrency and a token-bucket admission controller enforce the actual provider limit.

F. Engagement and Voluntary Data Contribution

Consistent diary logging is useful only when it reflects meaningful farm activity. The virtual pet (Bé Thóc) provides lightweight feedback based on streaks, completed reminders, and recent activity. Its state is rule-based and is used to adapt tone rather than to infer crop health. Gamification rewards are capped and deduplicated so that the system does not encourage repetitive or fabricated entries.

Farm Snap extends the workflow with optional community photo sharing. Contribution to model-improvement datasets is separate from posting: users must explicitly opt in, may withdraw consent for future use, and can use the sharing feature without agreeing to training use. Exact coordinates are not included in training exports by default; location is omitted or generalized to an approved administrative level. Only de-identified, quality-reviewed records with valid consent status are eligible for export. This design treats dataset creation as a governed research activity rather than a hidden by-product of engagement.

IV. System Architecture

A. Overview

Farmy follows a Decoupled Modern Web PWA / Hybrid Cloud architecture. The React Vite PWA client communicates with the NestJS backend exclusively over HTTPS REST. The backend implements Hexagonal Architecture across four layers: Gateway (controllers, webhooks), Application (guards, pipes, interceptors), Domain (business services, AI pipeline), and Infrastructure (MongoDB, pgvector, Redis, BullMQ, Cloudflare R2, Gemini API). Figure 1 illustrates the high-level topology.

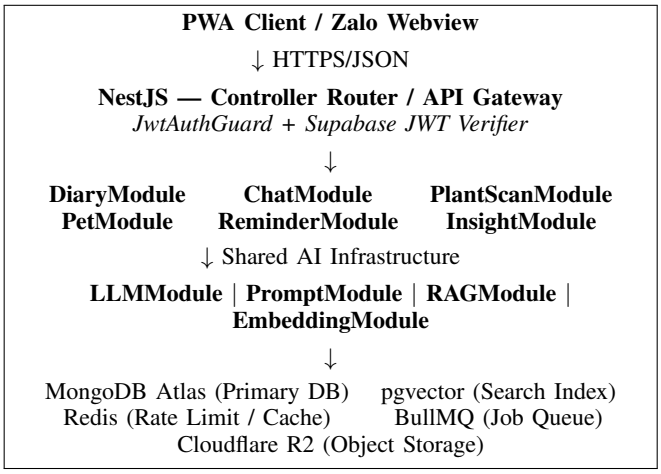

Fig. 1. Farmy high-level system architecture.

B. Database Layer

MongoDB Atlas is the **exclusive primary database** for all application data: user profiles, diary entries, chat sessions, pet states, plant scan records, weekly insights, reminders, audit logs, and knowledge content. pgvector (a PostgreSQL extension) serves solely as a **vector search index**, containing one table (`embeddings`) with columns (`id, source_id, source_type, embedding vector(768), metadata, is_active, created_at`) and no business data.

This separation provides two properties: (1) if the pgvector index is lost, it can be fully reconstructed from MongoDB by re-running the embedding pipeline; (2) all CRUD operations in application code target MongoDB, preserving a single coherent source of truth.

pgvector was selected as a rebuildable search-index component after considering operational control and the desire to keep business records exclusively in MongoDB. Latency and total cost must be measured against current service offerings and the actual corpus before deployment; therefore, the architecture does not rely on fixed provider-pricing or latency claims.

C. Caching and Queue Architecture

Redis serves three roles: (1) rate-limit counters for Gemini API quota enforcement; (2) plant scan result caching keyed on perceptual hash; and (3) BullMQ broker state. BullMQ manages four priority queues: `llm_queue` (priority 1 for interactive chat, priority 2 for plant scans), `embed_queue` (priority 3), `reminder_queue` (priority 5), and `insight_queue` (priority 10, lowest). This hierarchy ensures interactive requests are never blocked by background batch operations.

D. Object Storage and File Security

Cloudflare R2 (S3-compatible API) hosts all binary assets. The backend issues short-lived pre-signed GET URLs (TTL = 1 hour) for client download and short-lived signed PUT URLs (TTL = 5 minutes) for client upload. File uploads pass two independent validation checks: MIME type filtering via Multer `fileFilter`, and magic-bytes verification using the `file-type` library to detect format spoofing.

E. Notification Architecture

The notification subsystem is designed behind a channel-agnostic interface. Candidate channels include PWA Web Push, Zalo template messaging, and email. Channel order

is configurable because availability, approval requirements, pricing, user consent, and delivery reliability may change. Reminder messages contain concise action summaries and a deep link to the relevant diary or scan record; sensitive diagnostic content is not placed in lock-screen notifications by default.

V. AI Architecture

Farmy separates AI concerns across four modules with well-defined interfaces, enabling independent development and testing.

A. LLMModule

LLMModule is the **singleton gateway** for all Gemini API calls. No other module invokes Gemini directly. The module exposes three methods: `complete()` for text generation via Gemini Flash, `completeVision()` for multimodal diagnosis, and `embed()` for 768-dimensional vector generation via `text-embedding-004`.

Rate limiting uses independent Redis counters or token buckets per provider operation. Limits are deployment configuration derived from current provider quotas. When the generation limit is saturated, interactive chat returns a fallback message and background jobs receive a rate-limit exception. Retry logic uses bounded exponential backoff before invoking the fallback path.

B. PromptModule

PromptModule implements a **Prompt-as-Code** approach: prompt construction is a pure, side-effect-free service accepting structured input and returning a versioned `BuiltPrompt` object. Three builder methods correspond to the three AI pipelines: `buildChatPrompt()`, `buildVisionPrompt()`, and `buildWeeklyInsightPrompt()`.

Determinism requirement: given identical inputs, builders produce byte-identical output. No internal use of `Date.now()`, `Math.random()`, or other non-deterministic operations is permitted.

Injection resistance: user messages and retrieved context are treated as untrusted data and inserted only into delimited prompt fields. Separate sanitization functions may remove known control-token patterns, but blocklists are supplementary rather than authoritative. Application code owns authorization, tool permissions, citation provenance, and output-schema validation; retrieved text cannot modify those controls.

Truncation: character limits are applied independently—user messages: 2,000 chars; RAG context: 6,000 chars; conversation history: 4,000 chars retaining the most recent six turns.

The `promptVersion` field in `BuiltPrompt` is passed to LLMModule for logging, enabling regression detection when templates change.

C. RAGModule

RAGModule implements context retrieval for Chat AI and Weekly Insight pipelines. The retrieval process is:

1) Embed the user query via `text-embedding-004` ($\mathbb{R}^{768}$).
2) Query the pgvector HNSW index using cosine similarity and application-configured candidate limits. Knowledge retrieval filters by source status and optional crop metadata. Diary retrieval additionally applies the non-bypassable filter `metadata.userId = authenticatedUserId`.
3) Receive `(source_id, source_type, chunk_id, score)` tuples only—no business content from pgvector.
4) Fetch authorized full documents from MongoDB and discard missing, deleted, or unauthorized records.
5) Optionally combine semantic score with lexical, crop, source-authority, and recency signals before assembling context within a configured budget.
6) Build citation provenance from the retrieved records in application code; the model is not trusted to invent source identifiers.

The Hybrid design simultaneously retrieves from two corpora: knowledge chunks (verified agronomic documents) and diary entries (user-authored records). This allows responses to reference both domain knowledge (e.g., disease biology) and personal farm context (e.g., a specific treatment applied 11 days prior).

Every returned context item carries application-generated provenance: `sourceType`, `sourceId`, `chunkId`, retrieval score, and an authorized display label. Citations shown to the user are rendered from this provenance after generation; free-form source identifiers emitted by the LLM are ignored.

TABLE II
RAG Retrieval Evaluation Targets

Metric	Definition	Target
Precision@6	Proportion of retrieved docs rated relevant by agronomist	≥ 0.70
Recall@6	Proportion of known-relevant docs retrieved	≥ 0.60
Context Relevance	Avg. cosine similarity: context vs. ground-truth answer	≥ 0.75

D. EmbeddingModule

EmbeddingModule manages the lifecycle of vector embeddings in pgvector. Embeddings are created asynchronously via BullMQ jobs, decoupled from the synchronous MongoDB write that precedes them (Dual Write pattern). If an embedding job fails permanently (after 3 retries with exponential backoff), the diary entry is safe in MongoDB—only search coverage is temporarily reduced. A nightly gap-fill cron identifies entries without active embeddings and re-queues their jobs.

Text chunking strategy by source type:

- **Diary notes** < 20 **chars**: skipped—semantically insufficient.
- **Diary notes 20–499 chars**: embedded as a single chunk.
- **Diary notes** ≥ 500 **chars**: sliding window, 300-char window, 100-char step, maximum 10 chunks.
- **Knowledge documents**: 500-char window, 150-char step, maximum 50 chunks per document.

When a diary entry changes, new embeddings are generated before the index pointer is switched. After the replacement rows are active, previous rows are marked inactive and later removed by a cleanup job. If regeneration fails, the previous active rows remain available, preserving retrieval coverage.

VI. Plant Disease Analysis

A. Validation Pipeline

The plant disease detection pipeline applies three sequential validation layers before invoking Gemini Vision, reducing unnecessary API calls and ensuring minimum image quality.

1. Rate Limit Check
Redis counter: 3 scans/day (free) / 10 scans/day (premium)

↓ PASS

2. Magic Bytes Validation
file-type library: must match JPEG / PNG / WebP header

↓ PASS

3. Blur Detection (Sharp.js)
Laplacian variance $< 100 \Rightarrow$ reject, prompt retake

↓ PASS

4. Perceptual Hash (pHash) Cache
Hamming distance < 10 within 7 days $\Rightarrow$ return cached result

↓ CACHE MISS

5. Cloudflare R2 Upload + Gemini Vision
Structured JSON output: diagnosis candidates, certainty, evidence quality, treatment cautions

↓

6. Persist to MongoDB + Return to Client

Fig. 2. Plant disease scan pipeline with upstream validation layers.

Blur detection: the Laplacian variance of the image is computed via an image-processing component. An initial threshold may be used to request a retake, but the value must be calibrated and reported on a labeled field-image validation set before being treated as a validated quality boundary.

Perceptual hash caching: a pHash is computed for each accepted image. Images with Hamming distance below 10 from a cached scan within the past 7 days return the cached diagnosis, eliminating duplicate API calls for the same or visually similar field conditions.

Magic-bytes verification: the binary header of each uploaded file is inspected independently of the declared MIME type to prevent format spoofing.

B. Gemini Vision Prompt

The vision prompt instructs Gemini to return a structured JSON response containing: `isPlant` (boolean), one or more diagnosis candidates, `diagnosticCertainty` (`low|medium|high`), an `evidenceQuality` object, visible symptoms, treatment options, safety cautions, PHI-related information when a specific registered product is identified, and a disclaimer. These values are model assessments, not calibrated probabilities. Low certainty, poor evidence quality, or conflicting candidates require a retake or expert-consultation warning. If `isPlant` is false, the pipeline returns a structured negative without a disease label.

C. Safety Guardrails

A post-processing function applies the BVTV (plant protection) guardrail: any treatment recommendation referencing a restricted or prohibited pesticide under Vietnamese regulation is flagged with an explicit safety alert. All diagnosis responses carry a disclaimer advising consultation with a local agricultural extension officer. This guardrail is implemented in application code rather than relying solely on model output. It is a risk-reduction measure and does not replace verification against current regulations or professional agronomic advice.

D. Future Direction

The MVP Gemini Vision approach provides immediate utility but faces latency (5–10 s round-trip), external API dependency, and potential output format inconsistency. Future work will fine-tune EfficientNet-B4 or MobileNetV3 on a Vietnamese field-condition dataset seeded through the Farm Snap data flywheel. Candidate architectures include:

- **EfficientNet-B4** [8]: strong per-class accuracy on multi-disease classification with compact model size.
- **MobileNetV3** [9]: optimized for mobile inference, enabling potential on-device deployment.
- **YOLO classification** [10]: single-pass real-time classification within a camera viewfinder.

VII. Personalized Farm Memory

A. From Generic to Personalized Advice

Generic agricultural chatbots answer: *"What disease affects rice during the tillering stage?"* Farmy answers: *"Given that your rice field was last treated for brown planthopper 11 days ago, soil moisture was low in four of the past seven diary entries, and humidity is elevated this week, what is the most appropriate intervention?"* This requires treating diary history as operational memory addressable by semantic query.

B. Diary as Farm Memory

Each diary entry captures: crop type, growth stage, free-text notes, photographs, watering and fertilization events, weather context, and geographic location. Upon creation, entries are persisted to MongoDB synchronously and enqueued for embedding asynchronously. Once embedded, entries are retrievable via cosine similarity against user queries in the RAG pipeline.

Diary data serves three distinct roles:

1) **UI History Layer**: chronological operational record in the diary list view.

2) **RAG Memory Layer**: embedded chunks retrieved as context for AI chat.
3) **Weekly Insight Source**: the past seven days of entries are synthesized into a weekly farm activity summary with forward-looking recommendations.

C. Pet State as Behavioral Context

The virtual pet mascot (Bé Thóc) maintains a rule-based mood derived from the farmer's diary streak and entry recency. Mood values (`happy`, `excited`, `neutral`, `sad`, `worried`) are injected into the chat system prompt, allowing the AI to adapt tone and include streak-appropriate encouragement. This creates a feedback loop: AI responses reinforce the consistent diary behavior that in turn enriches AI response quality.

D. Farm Snap Data Flywheel

Farm Snap is an instant photo-sharing feature with a dual purpose. Users experience it as a community feed where they share field photographs and receive peer reactions. A submission becomes eligible for model-improvement research only after explicit, revocable opt-in consent. The user supplies `cropType` and `condition` (healthy / issue / harvest / other). Location is omitted by default or coarsened to the minimum useful granularity; precise coordinates require a separate purpose and consent. Only de-identified, quality-reviewed samples may be exported, and withdrawal workflows must remove future eligibility.

VIII. Implementation

A. Frontend

The client is a PWA implemented in React 18, Vite, and TypeScript. Styling uses Tailwind CSS with Shadcn/UI primitives. State management uses Zustand for local state and TanStack Query for server state with cache invalidation. PWA capabilities (offline caching, installability) are implemented via `vite-plugin-pwa`. The application is mobile-first, targeting portrait-orientation smartphone use.

B. Backend

The backend is built on NestJS 10 with TypeScript strict mode. The module system enforces controller / service / DTO / repository separation within each feature domain. Input validation uses class-validator at the DTO layer with a global ValidationPipe configured for property whitelisting and non-whitelisted rejection. Supabase Auth owns identity and application sessions. The client obtains a Supabase session, and NestJS verifies the Supabase JWT for each protected request using a Passport strategy or equivalent verifier. The backend does not issue a second custom refresh-token family.

C. AI Layer Dependency Hierarchy

The AI module dependency order is:

$$\text{LLM} \leftarrow \text{Embedding} \leftarrow \text{RAG} \leftarrow \text{Features}. \tag{13}$$

PromptModule has no dependencies, making it independently unit-testable without mocking. LLMModule is at the base; no module above it invokes Gemini directly. This hierarchy prevents scattered API calls, centralizes quota management, and simplifies provider substitution.

D. Infrastructure

Docker Compose orchestrates local development with services for NestJS, MongoDB, PostgreSQL+pgvector, and Redis. A candidate deployment uses an application hosting provider, MongoDB Atlas, a PostgreSQL service with pgvector support, Redis, and Cloudflare R2. Final providers are selected after verifying current quotas, pricing, region availability, and operational requirements. CI/CD runs on GitHub Actions: lint $\rightarrow$ type-check $\rightarrow$ unit tests $\rightarrow$ integration tests (with pgvector, Redis, and MongoDB service containers) $\rightarrow$ build verification $\rightarrow$ staging deploy $\rightarrow$ smoke test $\rightarrow$ production deploy.

TABLE III
Infrastructure Stack

Layer	Technology	Role
Frontend	React 18 + Vite PWA	Client application
Backend	NestJS 10 + TypeScript	API server
Primary DB	MongoDB Atlas	All business data
Vector Index	pgvector (HNSW)	Similarity search only
Cache / Queue	Redis + BullMQ	Rate limit, job scheduling
Storage	Cloudflare R2	Binary assets
Auth	Supabase Auth	Identity provider
LLM	Gemini Flash	Text generation
Embedding	text-embedding-004	768-dim vectors
Vision	Gemini Vision	Crop disease diagnosis
Notification	Zalo ZNS / Web Push	User alerts

IX. Evaluation Plan

The evaluation is staged so that architectural correctness is established before user-facing effectiveness is assessed. Planned targets are treated as decision criteria, not as achieved results.

A. Stage 1: Software and Security Verification

Unit tests cover prompt builders, schema validators, ownership filters, retry behavior, and rule-based state transitions. Integration tests verify the two-step retrieval path from pgvector identifiers to MongoDB documents, including an explicit negative test showing that diary records belonging to another user cannot be retrieved. End-to-end tests cover diary creation, asynchronous indexing, chat retrieval, scan-to-diary linking, reminder generation, and consent withdrawal. Test counts and coverage percentages will be reported only from the submitted commit and CI logs.

B. Stage 2: Retrieval Evaluation

A benchmark set will be created from realistic Vietnamese agricultural questions paired with relevance judgments for both knowledge chunks and diary events. At least two reviewers will label relevance and resolve disagreements. Metrics include

Precision@k, Recall@k, mean reciprocal rank, and tenant-isolation violations. Ablations compare knowledge-only retrieval, diary-only retrieval, naive merged retrieval, and the proposed provenance-aware dual-corpus approach. Recency weighting and metadata filters will be reported as explicit configurations.

C. Stage 3: Response Quality and Groundedness

Answers will be evaluated on correctness, relevance, evidence coverage, citation accuracy, uncertainty communication, and safety. Reviewers receive the question, retrieved evidence, and response, but not the system variant when feasible. Automatic model-based grading may be used for exploratory analysis, but primary claims will rely on human judgments from qualified agricultural reviewers. Citation precision is measured by checking whether each cited source supports the associated statement; unsupported claims are counted separately from stylistic weaknesses.

D. Stage 4: Plant-Observation Evaluation

The image workflow will be evaluated on a consented field-image set with reference labels provided or verified by qualified reviewers. The protocol reports dataset composition, crop and condition coverage, image acquisition conditions, and class imbalance. Metrics include candidate-level precision and recall, severe-condition false-negative rate, evidence-quality rejection accuracy, and the frequency with which low-certainty cases are appropriately referred. Because multimodal self-assessment is not calibrated probability, no claim will equate the returned certainty level with statistical confidence.

E. Stage 5: Usability Study

A pilot study will recruit participants using documented inclusion criteria and informed consent. Tasks include creating a diary entry, asking a question that depends on past history, submitting a plant image, and reviewing a reminder. Measures include task completion, completion time, observed errors, the System Usability Scale, and semi-structured feedback. Sample size, recruitment location, ethics approval requirements, and participant characteristics will be stated in the final paper rather than assumed in advance.

F. Reporting and Reproducibility

The final submission will distinguish measured results from design targets. It will identify the software revision, prompt version, embedding model, corpus version, evaluation date, and reviewer protocol. Negative and inconclusive findings will be retained. Where data cannot be publicly released because it contains farm or location information, the paper will publish schemas, synthetic examples, evaluation scripts, and aggregate statistics sufficient to inspect the methodology.

X. Limitations

Prototype status. The current paper describes an implementable architecture and evaluation plan. It does not yet establish agronomic effectiveness, sustained user adoption, or production reliability.

Model fallibility. Retrieval does not eliminate hallucination. A response can misinterpret relevant evidence, overlook contradictory information, or produce unsafe specificity. The assistant must therefore be positioned as decision support rather than an autonomous agronomic authority.

Knowledge and regulatory maintenance. Recommendations are bounded by the coverage and freshness of the curated corpus. Pesticide restrictions, labels, and regional practices change over time; stale documents can be harmful even when retrieval is technically correct.

Field-image ambiguity. Similar symptoms may arise from disease, nutrient deficiency, weather stress, or chemical injury. A single photograph may not contain the context needed to distinguish these causes, and ordinal model certainty is not calibrated probability.

Privacy and representativeness. Diary and image data may reveal location, production practices, and economic conditions. Consent, retention, access control, and de-identification require continuing governance. Voluntary Farm Snap contributions may also overrepresent engaged users and visually obvious conditions.

Infrastructure dependence. The prototype depends on external model and hosting services whose quotas, interfaces, regional availability, and prices may change. Cost-constrained operation is a deployment objective, not a guaranteed property.

XI. Future Work

Farm memory expansion. Future RAG retrievals will include plant scan results and weekly insight summaries alongside diary entries, creating a comprehensive personal farm memory spanning all data collected by the application.

Fine-tuned disease detection. After a sufficiently large, diverse, consented, and independently reviewed field-image corpus is available, task-specific models such as EfficientNet-B4 or MobileNetV3 can be evaluated against the multimodal baseline. Required sample size will be determined through learning curves rather than a fixed undocumented threshold. On-device inference is a possible direction if accuracy, model size, and device constraints permit.

Regional disease knowledge graph. Aggregated anonymized patterns from diary and scan records could support a regional crop health knowledge graph—mapping disease prevalence by crop type, geography, and season—enabling proactive alert recommendations and dynamically weighted RAG retrieval.

Agricultural marketplace integration. A marketplace module linking sales records to the diary and crop management history would provide end-to-end farm-to-market traceability. Buyers could verify agronomic practice records and AI-diagnosed disease history as part of quality certification.

Multi-agent advisory architecture. Specialized agents for soil management, pest control, irrigation scheduling, and market timing, coordinated by an orchestrating agent, could improve response specificity through targeted context routing.

Local LLM fine-tuning. Dependence on general-purpose LLMs introduces quality degradation for localized Vietnamese

agricultural terminology and regional cultivation practices. Open-weight language models can be evaluated as alternatives to hosted services. Any fine-tuning study should use licensed data, held-out evaluation, and comparison against retrieval-only adaptation before claiming improved domain performance.

XII. Conclusion

This paper presented the design of Farmy, a diary-centered agricultural support system that treats a farmer's prior records as private, reusable retrieval memory. The architecture combines curated domain documents with authenticated-user diary entries while preserving ownership filters and source provenance. Plant observations are connected to diaries, reminders, and follow-up conversation rather than being isolated one-time predictions. Prompt construction and citation metadata are separated from model invocation so that important behavior can be tested and audited.

The paper deliberately limits its claims. It does not demonstrate that the system improves yield, replaces expert diagnosis, or is ready for production deployment. Instead, it contributes a coherent prototype architecture and an evaluation plan capable of testing the central hypothesis: whether combining verified agricultural material with user-specific farm history produces answers that are more relevant and better grounded than generic assistance alone. The next version of the work should report reproducible retrieval experiments, expert review of generated advice, field-image evaluation, and a consented usability study. These results will determine whether the proposed integration provides sufficient value and safety to justify broader deployment.

Acknowledgment

The authors thank the Faculty of Software Engineering at FPT University for academic guidance during the SDN392 Capstone Project. Any external agricultural reviewers or study participants will be acknowledged in a later version only after their actual contribution and consent have been confirmed.

References

[1] World Bank, "Employment in agriculture (% of total employment), Viet Nam," World Development Indicators, based on ILO modeled estimates. Accessed: Jun. 2026.

[2] World Bank, "Vietnam Poverty and Equity Brief," The World Bank Group, Washington, D.C., 2023.

[3] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep learning in agriculture: A survey," *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[5] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.

[6] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics through machine learning and crowdsourcing," *arXiv preprint arXiv:1511.08060*, 2015.

[7] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, "PlantDoc: A dataset for visual plant disease detection," in *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*, 2020, pp. 249–253.

[8] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, vol. 97, 2019, pp. 6105–6114.

[9] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 1314–1324.

[10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.

[11] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[12] M. G. Selvaraj, A. Vergara, H. Ruiz, N. Safari, S. Elayabalan, W. Ocimati, and G. Blomme, "AI-powered banana diseases and pest detection," *Plant Methods*, vol. 15, no. 1, pp. 1–11, 2019.

[13] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[14] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation," arXiv:2305.06983, 2023.

[15] Y. Zhang et al., "ShizishanGPT: An agricultural large language model integrating tools and retrieval-augmented generation," arXiv:2409.13537, 2024.

[16] M. Fanuel, M. N. Mahmoud, C. C. Marshal, V. Lakhotia, B. Dari, K. Roy, and S. Zhang, "AgriRegion: Region-aware retrieval for high-fidelity agricultural advice," arXiv:2512.10114, 2025.